Bit Manipulation Study Guide

1. Introduction to Bit Manipulation

Bit manipulation is the act of algorithmically manipulating bits or other pieces of data shorter than a byte. In Java, bit manipulation involves working with individual bits of integers using bitwise operators. This technique is fundamental in computer science and is used to optimize programs, solve specific algorithmic problems, and work with low-level programming tasks.

Bit manipulation operates on the binary representation of numbers. For example, the number 5 in binary is 101, and the number 3 in binary is 011. Understanding how numbers are represented in binary is crucial for effective bit manipulation.

2. Bitwise Operations

Java provides several bitwise operators that allow you to manipulate individual bits:

AND Operator (&)

The AND operator returns 1 only when both bits are 1.

```java
int a = 5;  // Binary: 101
int b = 3;  // Binary: 011
int result = a & b;  // Binary: 001, Decimal: 1

System.out.println("5 & 3 = " + result); // Output: 1
```

OR Operator (|)

The OR operator returns 1 when at least one bit is 1.

```java
int a = 5;  // Binary: 101
int b = 3;  // Binary: 011
int result = a | b;  // Binary: 111, Decimal: 7

System.out.println("5 | 3 = " + result); // Output: 7
```

XOR Operator (^)

The XOR operator returns 1 when bits are different.

```java
int a = 5;  // Binary: 101
int b = 3;  // Binary: 011
int result = a ^ b;  // Binary: 110, Decimal: 6

System.out.println("5 ^ 3 = " + result); // Output: 6
```

Left Shift Operator (<<)

Shifts bits to the left, effectively multiplying by 2^n.

```java
int a = 5;  // Binary: 101
int result = a << 2;  // Binary: 10100, Decimal: 20

System.out.println("5 << 2 = " + result); // Output: 20
```

Right Shift Operator (>>)

Shifts bits to the right, effectively dividing by 2^n.

```java
int a = 20;  // Binary: 10100
int result = a >> 2;  // Binary: 101, Decimal: 5

System.out.println("20 >> 2 = " + result); // Output: 5
```

NOT Operator (~)

The NOT operator flips all bits (1 becomes 0, 0 becomes 1).

```java
int a = 5;  // Binary: 101
int result = ~a;  // Flips all bits

System.out.println("~5 = " + result); // Output: -6 (due to two's complement)
```

3. Application of Bitwise Operations

Bitwise operations have numerous practical applications in programming:

Performance Optimization: Bitwise operations are faster than arithmetic operations for certain calculations.

```java
// Instead of multiplying by 2
int doubled = number * 2;
// Use left shift (faster)
int doubled = number << 1;

// Instead of dividing by 2
int halved = number / 2;
// Use right shift (faster)
int halved = number >> 1;
```

Checking Even/Odd Numbers: Using AND operation with 1.

```java
public static boolean isEven(int number) {
    return (number & 1) == 0;
}

public static boolean isOdd(int number) {
    return (number & 1) == 1;
}
```

Swapping Two Numbers: Without using a temporary variable.

```java
public static void swap(int a, int b) {
    System.out.println("Before: a = " + a + ", b = " + b);

    a = a ^ b;
    b = a ^ b;
    a = a ^ b;

    System.out.println("After: a = " + a + ", b = " + b);
}
```

Finding Single Number: In an array where every element appears twice except one.

```java
public static int findSingle(int[] arr) {
    int result = 0;
    for (int num : arr) {
        result ^= num;
    }
    return result;
}
```

4. Basics of Bit Manipulation

a) Get Bit

To get the value of a bit at a specific position, use the AND operator with a mask.

```java
public static int getBit(int number, int position) {
    return (number >> position) & 1;
}

// Example usage
int number = 5;  // Binary: 101
int bit0 = getBit(number, 0);  // Returns 1
int bit1 = getBit(number, 1);  // Returns 0
int bit2 = getBit(number, 2);  // Returns 1

System.out.println("Bit at position 0: " + bit0);
System.out.println("Bit at position 1: " + bit1);
System.out.println("Bit at position 2: " + bit2);
```

b) Set Bit

To set a bit at a specific position to 1, use the OR operator with a mask.

```java
public static int setBit(int number, int position) {
    return number | (1 << position);
}

// Example usage
int number = 5;  // Binary: 101
int result = setBit(number, 1);  // Binary: 111, Decimal: 7

System.out.println("Original: " + number);
System.out.println("After setting bit 1: " + result);
```

c) Clear Bit

To clear a bit at a specific position (set it to 0), use the AND operator with an inverted mask.

```java
public static int clearBit(int number, int position) {
    return number & ~(1 << position);
}

// Example usage
int number = 7;  // Binary: 111
int result = clearBit(number, 1);  // Binary: 101, Decimal: 5

System.out.println("Original: " + number);
System.out.println("After clearing bit 1: " + result);
```

d) Toggling Bit

To toggle a bit at a specific position (flip its value), use the XOR operator with a mask.

```java
public static int toggleBit(int number, int position) {
    return number ^ (1 << position);
}

// Example usage
int number = 5;  // Binary: 101
int result1 = toggleBit(number, 1);  // Binary: 111, Decimal: 7
int result2 = toggleBit(result1, 1);  // Binary: 101, Decimal: 5

System.out.println("Original: " + number);
System.out.println("After toggling bit 1: " + result1);
System.out.println("After toggling bit 1 again: " + result2);
```

e) Unset Bit

Unset bit is the same as clear bit - setting a specific bit to 0.

```java
public static int unsetBit(int number, int position) {
    return number & ~(1 << position);
}

// Alternative method using XOR (only if the bit is currently 1)
public static int unsetBitXOR(int number, int position) {
    if (getBit(number, position) == 1) {
        return number ^ (1 << position);
    }
    return number;
}

// Example usage
int number = 15;  // Binary: 1111
int result = unsetBit(number, 2);  // Binary: 1011, Decimal: 11

System.out.println("Original: " + number);
System.out.println("After unsetting bit 2: " + result);
```

Quiz Questions

Question 1: What is the result of 6 & 4 in decimal? a) 2 b) 4 c) 6 d) 10

Answer: b) 4 (Binary: 110 & 100 = 100 = 4)

Question 2: Which bitwise operator is used to set a specific bit to 1? a) AND (&) b) OR (|) c) XOR (^) d) NOT (~)

Answer: b) OR (|)

Question 3: What does the expression `number << 3` do? a) Divides number by 3 b) Multiplies number by 3 c) Multiplies number by 8 d) Adds 3 to number

Answer: c) Multiplies number by 8 (Left shift by 3 positions = multiply by 2^3 = 8)

Question 4: To check if a number is odd using bit manipulation, which expression should you use? a) `(number & 1) == 0` b) `(number & 1) == 1` c) `(number | 1) == 1` d) `(number ^ 1) == 0`

Answer: b) `(number & 1) == 1`

Question 5: What is the result of 5 ^ 5? a) 0 b) 5 c) 10 d) 25

Answer: a) 0 (Any number XOR with itself equals 0)

Practice Coding Questions

Question 1: Write a Java method that counts the number of 1s in the binary representation of a given integer.

Question 2: Create a program that checks if a given number is a power of 2 using bit manipulation techniques.

Question 3: Write a method that finds the position of the rightmost set bit (1) in a given integer. Return -1 if no set bit is found.

Question 4: Implement a function that reverses the bits of a given integer. For example, if the input is 12 (binary: 1100), the output should be 3 (binary: 0011).

Question 5: Write a program that finds two non-repeating elements in an array where every other element repeats exactly twice.